

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE**

SPECIALIZATION Computer Science

DIPLOMA THESIS

Pseudo++: A Comprehensive Pseudocode Toolchain for Romanian High School Computer Science

Supervisor
Assoc. Prof. Rareș Florin Boian, PhD

Author
Bujor Mihai Alexandru

2026

Abstract

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Motivation	2
1.3 Related Work	3
1.4 Contributions	3
1.5 Thesis Structure	4
2 Related Work	5
2.1 General-Purpose Online Code Editors	5
2.2 Pseudocode-Specific Tools	5
2.3 Educational Language Interpreters	5
2.4 Transpilation in Educational Contexts	5
2.5 Comparison with the Present Work	5
3 Theoretical Background	6
3.1 Monaco Editor and Monarch Tokenisation	6
3.1.1 Monaco Editor	6
3.1.2 The Monarch Tokeniser Model	7
3.1.3 Monarch vs. Tree-sitter	7
3.2 Syntactic Analysis and Incremental Parsing	7
3.2.1 Classical Parsing Approaches	7
3.2.2 Tree-sitter and Incremental GLR Parsing	8
3.2.3 Concrete vs. Abstract Syntax Trees	8
3.2.4 Error Recovery	8
3.3 Interpretation and Transpilation	9
3.3.1 Interpreters	9
3.3.2 Transpilation	9
3.3.3 The Pseudo++ Processing Pipeline	10
3.4 Equivalence of Repetitive Control Structures	10

3.4.1	Structured Programming and the Böhm-Jacopini Theorem . .	10
3.4.2	Semantics of the Four Loop Forms	11
3.4.3	Transformation Rules	11
3.5	WebAssembly	12
3.5.1	Overview	12
3.5.2	Emscripten	12
3.5.3	Application in Pseudo++	12
3.6	Statement-Level Step-Through Debugging	13
3.6.1	Interactive Debugging in Educational Contexts	13
3.6.2	Per-Step State Capture	13
3.6.3	Condition Visualisation	14
4	The Pseudo++ Toolchain	15
4.1	System Architecture	15
4.2	Language Grammar	16
4.2.1	Statement Forms	17
4.2.2	Expression Hierarchy	17
4.3	Source Normalisation	18
4.4	Parsing and CST Construction	19
4.5	Interpreter	20
4.5.1	Value Representation	20
4.5.2	Execution Environment	21
4.5.3	Expression Evaluation	21
4.5.4	Control Flow	22
4.6	Debugger	23
4.7	Transpiler	24
4.7.1	Code Generation for C, C++, and Pascal	24
4.7.2	Loop Equivalence Transformations	25
4.8	I/O Abstraction and WebAssembly Integration	26
4.8.1	I/O Abstraction	26
4.8.2	WebAssembly Bridge	26
4.9	Web Editor	27
4.10	VSCode Extension	27
5	Evaluation	29
5.1	Testing Methodology	29
5.2	Unit Testing	30
5.3	Integration Testing on Baccalaureate Examination Problems	30
5.3.1	Dataset Description	30
5.3.2	Results	31

5.4	Transpiler Correctness Verification	31
5.5	Loop Equivalence Semantic Preservation	31
5.6	Performance Analysis	32
5.7	Discussion	33
6	Conclusions	34
6.1	Summary	34
6.2	Limitations	34
6.3	Future Work	34
	Bibliography	35

Introduction

The Romanian national baccalaureate examination in Computer Science requires candidates to read, write, and reason about programs expressed in a standardised pseudocode dialect [13]. This dialect is defined by the Ministry of Education and uses Romanian keywords (*citește, scrie, dacă, cât timp, pentru, repetă*), Unicode mathematical symbols for assignment and comparison ($\leftarrow, \leq, \geq, \neq$), and a visual block structure built from box-drawing characters ($\top, \mid, \bot, \blacksquare$). A representative fragment from the 2020 baccalaureate training set is shown below:

Despite being the language of instruction for secondary school CS in Romania, this dialect has no standard execution environment. Students preparing for the exam must mentally simulate the execution of each program, tracing variable values by hand on paper. This manual process is error-prone, time-consuming, and

provides no feedback on whether a solution is correct. Teachers face the same limitation when preparing or grading exam problems.

The dialect cannot be fed directly to any general-purpose language tool. Its Unicode symbols and Romanian keywords are not valid in C, C++, Pascal, or Python. General pseudocode interpreters designed for English-language curricula do not recognise the Romanian syntax. No existing tool supports the full feature set of the dialect: all four loop constructs (`cât timp`, `pentru`, `repetă...până când`, `execută...cât timp`), the floor-division operator, the square-root symbol, and the visual block-delimiter notation.

1.2 Motivation

The absence of executable tooling creates a concrete gap between the way students *learn* algorithms and the way they *verify* them. A student who writes a sorting or search algorithm in the baccalaureate pseudocode has no way to check whether it produces the correct output for a given input without executing it manually — a process that is both slow and susceptible to arithmetic errors.

Closing this gap motivates several specific capabilities, consistent with the broader goal of equipping students with tools that make algorithmic thinking *executable* [16]:

- **Execution.** Running a pseudocode program and observing its output is the most direct form of correctness feedback. An interpreter that accepts the canonical notation gives students an interactive scratchpad for testing their solutions.
- **Step-through debugging.** Watching variable values change statement by statement builds intuition about algorithm behaviour and makes it easier to pinpoint logic errors. A debugger tailored to the pseudocode dialect removes the need for manual trace tables.
- **Transpilation.** The baccalaureate exam also tests knowledge of C++ and Pascal. A transpiler that converts pseudocode to these languages lets students validate an algorithm in pseudocode and then produce a compilable implementation, reinforcing the connection between the two representations.
- **Accessibility.** A browser-based editor requires no installation and works on any device, which is important in a school context where students may not have administrator rights on lab computers.

Beyond individual students, a standard toolchain benefits the broader community: teachers can construct exam problems with known correct outputs, and curriculum authors can publish executable examples alongside written explanations.

1.3 Related Work

Several tools address adjacent problems. General pseudocode environments such as *PSeInt* [14] (a widely-used pseudocode interpreter for Spanish-speaking students) and *RAPTOR* [4] (a flowchart-based programming environment) support their respective notations but cannot parse Romanian keywords or Unicode symbols. Visual programming environments such as *Scratch* [15] target younger learners and are not suited to algorithmic notation at the secondary level. Browser-based coding platforms such as Replit and OnlineGDB accept standard languages (C++, Pascal, Python) but require the student to first translate the pseudocode manually. To the best of our knowledge, no tool exists that directly executes the Romanian baccalaureate pseudocode dialect. A detailed comparison with existing tools is provided in Chapter 2.

1.4 Contributions

This thesis presents **Pseudo++**, a complete open-source toolchain for the Romanian baccalaureate pseudocode dialect. The specific contributions are:

1. **A formal grammar for the dialect.** A Tree-sitter grammar (written in JavaScript) that precisely captures the syntax of the Romanian baccalaureate pseudocode, including all four loop constructs, operator precedence, and the visual block-delimiter notation.
2. **A two-pass source normaliser (linter).** A module that maps the canonical Unicode notation to an ASCII representation accepted by the parser, and then re-indents the source structurally, making the toolchain robust to the formatting variations that appear in practice.
3. **A tree-walking interpreter.** A C implementation that executes normalised pseudocode using an explicit execution stack and a dispatch table of per-statement step functions, supporting all language constructs including nested loops, conditionals, and dynamic typing.
4. **A snapshot-based step debugger.** An extension of the interpreter that captures the full runtime state before each statement, enabling variable inspection and step-through execution from a browser front end.
5. **A two-pass transpiler.** A module that converts pseudocode to equivalent C, C++, or Pascal source code, using a first pass for variable-type inference and a second pass for code emission.

6. **A loop-equivalence transformation module.** An analysis and rewrite module that converts any loop construct (`pentru, cât timp, repetă, execută...cât timp`) into any of the other three while preserving semantics.
7. **A browser-based web editor.** A single-page application embedding the Monaco editor with a custom Monarch syntax-highlighting grammar, wired to the C library compiled to WebAssembly via Emscripten, providing execution, debugging, transpilation, and equivalence views without any server-side component.
8. **A Visual Studio Code extension.** A standalone extension that provides syntax highlighting for `.pseudo` files via a TextMate grammar, with no dependency on the C library.
9. **A native command-line interface.** A CLI binary with subcommands for running, linting, parsing, transpiling, and computing loop equivalences, suitable for scripting and batch testing.

All components share a single C core library that is compiled both to a native binary (for the CLI) and to a WebAssembly module (for the web editor), ensuring that the browser and terminal interfaces exercise identical logic.

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 surveys related work, comparing Pseudo++ with existing pseudocode tools, educational programming environments, and language-implementation techniques. Chapter 3 covers the theoretical background: the Tree-sitter parsing framework, the Monaco editor and its Monarch grammar formalism, the WebAssembly compilation target, and the distinction between concrete and abstract syntax trees. Chapter 4 describes the design and implementation of each component of the Pseudo++ toolchain in detail. Chapter 5 presents an evaluation of the toolchain, including correctness testing against official baccalaureate problems, performance benchmarks, and a discussion of limitations. Chapter 6 summarises the contributions, reflects on design decisions, and outlines directions for future work.

Chapter 2

Related Work

2.1 General-Purpose Online Code Editors

2.2 Pseudocode-Specific Tools

2.3 Educational Language Interpreters

2.4 Transpilation in Educational Contexts

2.5 Comparison with the Present Work

Chapter 3

Theoretical Background

This chapter presents the theoretical foundations underpinning the Pseudo++ toolchain. It covers the Monaco editor and its Monarch tokenisation model used for syntax highlighting, the mechanics of incremental parsing via Tree-sitter, the conceptual distinctions between interpretation and transpilation together with Pseudo++’s bifurcated pipeline, the structured-programming basis for loop equivalence transformations, the WebAssembly execution model, and the snapshot-based approach used by the reversible debugger.

3.1 Monaco Editor and Monarch Tokenisation

3.1.1 Monaco Editor

Monaco Editor [11] is the open-source code-editing component that powers Visual Studio Code. It is written in TypeScript and runs entirely in the browser, providing a full-featured editing experience — line numbers, bracket matching, glyph margins, decorations, and theming — without a server-side component.

Monaco separates two language-related concerns:

- **Tokenisation** — assigning a visual token class (keyword, string, operator, comment, ...) to each contiguous run of characters for the purpose of syntax highlighting.
- **Language services** — richer, semantic operations such as auto-completion, hover information, and diagnostics, which require a deeper understanding of the language structure.

Pseudo++ uses Monaco for its web editor, registering a custom language (`pseudocode`) and providing both a Monarch tokeniser (Section 3.1.2) and a language configuration (bracket pairs, auto-closing, comment tokens).

3.1.2 The Monarch Tokeniser Model

Monarch [12] is Monaco’s declarative tokeniser specification language. A Monarch grammar is a JavaScript object describing a *finite-state transducer*: it has a set of *states*, and each state contains an ordered list of *rules* of the form $(regex, action)$.

The tokeniser processes the input line by line. In state q , it tries each rule’s regex against the current position; the first match fires the associated action, which emits a token class and optionally transitions to a new state. This gives Monarch the expressive power of a *regular language* per state, with the ability to represent richer patterns (e.g. multi-line strings) through state transitions.

For the Pseudo++ language, the single `root` state suffices. Rules recognise, in priority order: comments (`# . . .`), string literals, numbers, the swap and assignment operators (`<->`, `<-`), comparison operators, the floor brackets (`[, ]`), and finally identifiers — which are dispatched through case-insensitive keyword tables (`controlKeywords`, `flowKeywords`, `builtinFunctions`, `logicalOperators`) to assign the appropriate token class, falling back to `variable` for unknown identifiers.

3.1.3 Monarch vs. Tree-sitter

Monarch and Tree-sitter serve complementary but distinct roles in Pseudo++:

- **Monarch** runs entirely in JavaScript inside Monaco, is regex-based, and operates line by line. It is fast enough to re-tokenise the visible viewport on every keystroke, producing token classes consumed directly by Monaco’s renderer. It has no notion of nesting, scope, or grammatical structure.
- **Tree-sitter** runs as a WebAssembly module compiled from C, performs full incremental GLR parsing, and produces a concrete syntax tree (CST) encoding the complete grammatical structure of the program. It is used exclusively by the interpreter and transpiler for execution and code generation.

The two tools are never in conflict: Monarch makes the editor *look* right; Tree-sitter makes execution *work* right.

3.2 Syntactic Analysis and Incremental Parsing

3.2.1 Classical Parsing Approaches

Given a grammar G and an input string w , a *parser* determines whether $w \in L(G)$ and, if so, constructs a parse tree witnessing the derivation. Two broad families of algorithms exist:

- **Top-down parsers** (e.g. $LL(k)$, recursive descent) expand the start symbol top-down. They are simple to implement and produce intuitive error messages, but require the grammar to be left-recursion-free and left-factorable.
- **Bottom-up parsers** (e.g. $LR(k)$, LALR(1)) shift tokens onto a stack and reduce them by grammar rules. They handle a strictly larger class of grammars than LL parsers and are the basis for most production parser generators (`yacc`, `bison`, `menhir`).

3.2.2 Tree-sitter and Incremental GLR Parsing

Tree-sitter [2] is an open-source parsing framework that generates Generalised LR (GLR) parsers from a grammar described in a JavaScript DSL. Its defining feature is *incremental re-parsing*: when the source text changes, the parser does not restart from scratch but instead re-uses unchanged subtrees from the previous parse tree.

Formally, let T be a concrete syntax tree (CST) for source s , and let s' be the result of applying an edit δ to s . Tree-sitter maintains a set of *valid subtrees* — nodes whose byte ranges are unaffected by δ — and constructs T' by re-parsing only the minimal affected region, grafting in the cached subtrees. The amortised complexity of an edit of size k is $O(k \log n)$ where $n = |s|$ [1]. In Pseudo++, Tree-sitter is invoked each time the user requests execution or transpilation, giving an always up-to-date CST without redundant full re-parses.

3.2.3 Concrete vs. Abstract Syntax Trees

Tree-sitter produces *concrete syntax trees* (CSTs), which contain every token including punctuation and delimiter tokens. By contrast, an *abstract syntax tree* (AST) retains only semantically meaningful nodes.

Pseudo++ works directly on the CST exposed by Tree-sitter’s C API. Node types are string-tagged (`ts_node_type`), and named fields (e.g. `condition`, `var`, `start`) allow the interpreter and transpiler to locate child nodes by role rather than by index. This design avoids a separate lowering pass to an AST while retaining the benefits of a typed node API.

3.2.4 Error Recovery

Tree-sitter employs a built-in error-recovery strategy: when the parser encounters an unexpected token, it inserts `ERROR` and `MISSING` nodes into the tree rather than aborting. The resulting tree is always complete (every byte of input appears in some node) and serves as a basis for incremental syntax-error diagnostics. Pseudo++’s

linter traverses the tree looking for these sentinel nodes and reports their location to the user.

3.3 Interpretation and Transpilation

3.3.1 Interpreters

An *interpreter* is a program that directly executes a source program without producing standalone machine code. Interpreters can be categorised by the intermediate representation they operate on:

1. **Tree-walking interpreters** traverse the AST/CST and evaluate each node in turn. They are simple to implement and straightforward to extend, at the cost of performance compared to bytecode approaches.
2. **Bytecode interpreters** first compile to an intermediate bytecode (e.g. CPython's `.pyc`), then execute it in a virtual machine. This separation allows optimisations such as constant folding.
3. **JIT compilers** (e.g. LuaJIT, V8) profile execution and dynamically compile hot paths to native machine code.

Pseudo++ uses a *stack-based tree-walking interpreter*: an explicit execution stack mirrors the nesting of syntactic constructs, converting what would otherwise be recursive C calls into an iterative state machine. Each stack frame carries a *phase* counter that encodes progress through the frame's sub-steps, making the execution granularity uniform — exactly one statement per step — which is a prerequisite for the debugger's snapshot mechanism (Section 3.6).

3.3.2 Transpilation

Transpilation (source-to-source compilation) is a form of compilation where the target is another high-level language. Well-known examples include TypeScript-to-JavaScript (`tsc`), Sass-to-CSS, and Babel's ES2022-to-ES5 lowering. In the educational domain, transpilation from a didactic notation to a production language serves as a scaffolding device: students write in familiar pseudocode and observe the idiomatic equivalent in C or Pascal.

Pseudo++'s transpiler performs a single-pass tree walk over the CST, emitting target-language code for each node. A pre-pass (`transpiler_collect`) scans all assignment targets to produce the forward variable declarations required by C and Pascal, since both languages mandate that all variables are declared before use.

3.3.3 The Pseudo++ Processing Pipeline

After parsing, the CST is passed to one of two independent backends depending on the user's intent. Figure 3.1 shows the complete pipeline.

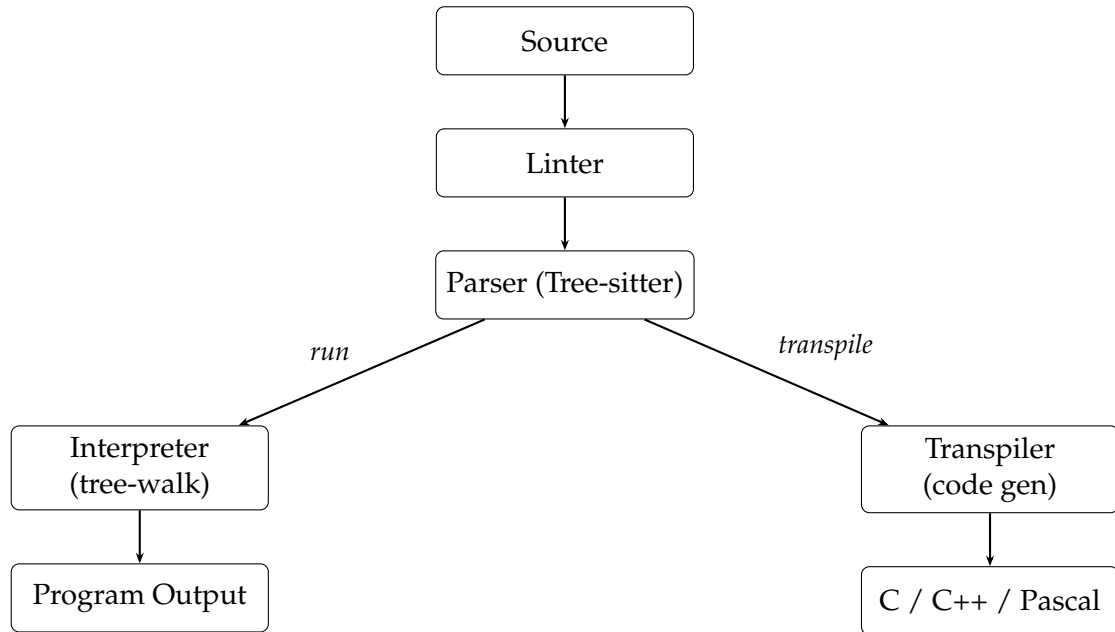


Figure 3.1: The Pseudo++ processing pipeline. After normalisation and parsing, the CST is handed to either the interpreter or the transpiler depending on the requested operation.

3.4 Equivalence of Repetitive Control Structures

3.4.1 Structured Programming and the Böhm-Jacopini Theorem

The theoretical basis for loop equivalence lies in *structured programming theory*. Böhm and Jacopini proved in 1966 [3] that any computable function can be expressed using only three control-flow primitives: sequence, selection (*if*), and a single repetition construct. This result, together with Dijkstra's influential argument against unstructured jumps [5], established the primacy of structured loops in procedural programming. A practical consequence is that the four loop forms commonly found in procedural languages — *for*, *while*, *do-while*, and *repeat-until* — are mutually interconvertible: any loop can be rewritten using any other form, at the cost of at most a guard condition or an unrolled first iteration.

This is not merely an academic curiosity. The Romanian high-school computer science curriculum [13] explicitly teaches all four loop forms and expects students to identify and apply these transformations in examination problems. Providing an automated, interactive converter therefore has direct pedagogical value.

3.4.2 Semantics of the Four Loop Forms

Let B denote the loop body (a finite sequence of statements), C a Boolean condition, i a counter variable, a the initial value, b the bound, and $s > 0$ the step size. The four loops have the following denotational semantics:

- **for**: execute B for $i = a, a+s, \dots$ while $i \leq b$; zero or more iterations.
- **while**: while C , execute B ; zero or more iterations.
- **do-while**: execute B , then repeat while C ; at least one iteration.
- **repeat-until**: execute B , then repeat until C (i.e. while $\neg C$); at least one iteration.

The key structural distinction is between *pre-test* loops (*while*, *for*) that may execute zero times, and *post-test* loops (*do-while*, *repeat-until*) that always execute at least once. Crossing the boundary between the two families requires a guard.

3.4.3 Transformation Rules

The following semantics-preserving rewrite rules hold unconditionally:

while $\rightarrow$ **do-while** (and back).

$$\mathbf{while\ } C \ \mathbf{do\ } B \equiv \mathbf{if\ } C \ \mathbf{then\ } (\mathbf{do\ } B \ \mathbf{while\ } C)$$

$$\mathbf{do\ } B \ \mathbf{while\ } C \equiv B; \ \mathbf{while\ } C \ \mathbf{do\ } B$$

for $\rightarrow$ **while**.

$$\mathbf{for\ } i \leftarrow a, b, s \ \mathbf{do\ } B \equiv i \leftarrow a; \ \mathbf{while\ } i \leq b \ \mathbf{do\ } (B; i \leftarrow i + s)$$

repeat-until $\rightarrow$ **while** (and back).

$$\mathbf{repeat\ } B \ \mathbf{until\ } C \equiv B; \ \mathbf{while\ } \neg C \ \mathbf{do\ } B$$

By composing these four base rules, all twelve directed pairwise conversions among the four loop forms can be derived. Pseudo++'s equivalence module implements each conversion as a text-level rewrite over the CST, preserving the original source indentation.

3.5 WebAssembly

3.5.1 Overview

WebAssembly (Wasm) [9, 8] is a binary instruction format for a stack-based virtual machine, standardised by the W3C in 2019. Its design goals are portability, safety, and near-native execution speed in the browser. A WebAssembly module is a self-contained binary (`.wasm`) that exports and imports named functions; the host (typically a JavaScript runtime) provides imports and calls exports.

Key properties of the Wasm execution model:

- **Linear memory:** a contiguous, byte-addressable memory region whose size is a multiple of 64 KiB. The C heap and stack both reside in linear memory; pointer arithmetic is valid within this region.
- **Type safety:** all instructions are typed; the validator rejects ill-typed modules before execution.
- **Sandboxing:** a Wasm module cannot access host memory outside its own linear memory or perform arbitrary I/O; all interaction with the outside world happens through explicitly imported functions.
- **Determinism:** aside from NaN bit-patterns, Wasm execution is fully deterministic.

3.5.2 Emscripten

Emscripten [17] is an LLVM-based compiler toolchain that compiles C and C++ to WebAssembly together with a JavaScript glue layer. Given a C source file, Emscripten compiles it to LLVM IR via `clang`, lowers it to Wasm via `wasm-ld` and `wasm-opt`, and generates a JavaScript companion (`.js`) that instantiates the module, provides standard C library shims, and exposes exported C functions to JavaScript callers.

Functions to be exported must be annotated with `EMSCRIPTEN_KEEPALIVE` to prevent dead-code elimination, or listed via the `-sEXPORTED_FUNCTIONS` linker flag.

3.5.3 Application in Pseudo++

The Pseudo++ interpreter is written in C23 and compiled to Wasm via Emscripten. A thin bridge layer (`wasm/bridge.c`) exposes the interpreter lifecycle and I/O routines:

```
EMSCRIPTEN_KEEPALIVE runtime_t* runtime_create_buffered(void);
EMSCRIPTEN_KEEPALIVE exec_state_t runtime_step(runtime_t* rt);
EMSCRIPTEN_KEEPALIVE const char* runtime_pop_output(runtime_t* rt);
EMSCRIPTEN_KEEPALIVE void runtime_push_input(runtime_t* rt,
                                             const char* line);
```

The I/O abstraction layer (`io.buffered`) replaces the standard `stdio` calls with a queue-based mechanism: output is pushed into a string queue that JavaScript drains via `runtime_pop_output`, and input is delivered by JavaScript calling `runtime_push_input` before the interpreter requests it. This non-blocking design ensures the browser's event loop is never stalled by a long-running computation.

3.6 Statement-Level Step-Through Debugging

3.6.1 Interactive Debugging in Educational Contexts

Step-through debugging allows a student to observe a program's behaviour one statement at a time, inspecting variable values and control flow at each point. This is particularly valuable when learning programming: rather than reading static code, the student sees the program *in motion* — which variable changes, which branch is taken, how a loop counter evolves [16].

The ability to step *backwards* through execution history — so-called *omniscient* or *reversible* debugging [10, 7] — extends this further by letting the developer inspect state at any previous point without restarting the program. Pseudo++'s snapshot architecture (Section 3.3.3) provides the foundation for this capability at the C runtime level.

Pseudo++'s debugger exposes two user-facing operations:

- **Step Into** (one statement at a time): executes the next statement, highlights the corresponding source line in the editor, and updates the variable inspection panel.
- **Continue** (run to completion): resumes fast execution without step-by-step visibility, stopping only when the program finishes, encounters a runtime error, or needs user input.

3.6.2 Per-Step State Capture

At each Step Into, the runtime first serialises the complete program state into a `runtime_snapshot_t` structure before advancing execution. A snapshot records:

- the variable environment: name, value, and type for every variable currently in scope;
- the execution stack: frame type, phase counter, loop bounds, loop counter, loop variable name, and the identity of the current CST node for each active frame;
- auxiliary I/O state: the multi-variable read index and a pending-input flag;
- the current line number and execution-state enum.

The snapshot identifier is stored by the JavaScript `PseudoDebugger` class alongside the cached variable values and condition-evaluation metadata for that step. Variable values are cached on the JavaScript side so that the variable panel can be refreshed without an additional round-trip to the Wasm module.

3.6.3 Condition Visualisation

When the statement about to execute is a conditional or loop guard, the runtime additionally serialises the condition expression text and its boolean result into a small JSON object. The front-end displays this beneath the variable panel, letting the student see not only *which* branch was taken but *why* — the exact condition text and its evaluated truth value at that step.

Chapter 4

The Pseudo++ Toolchain

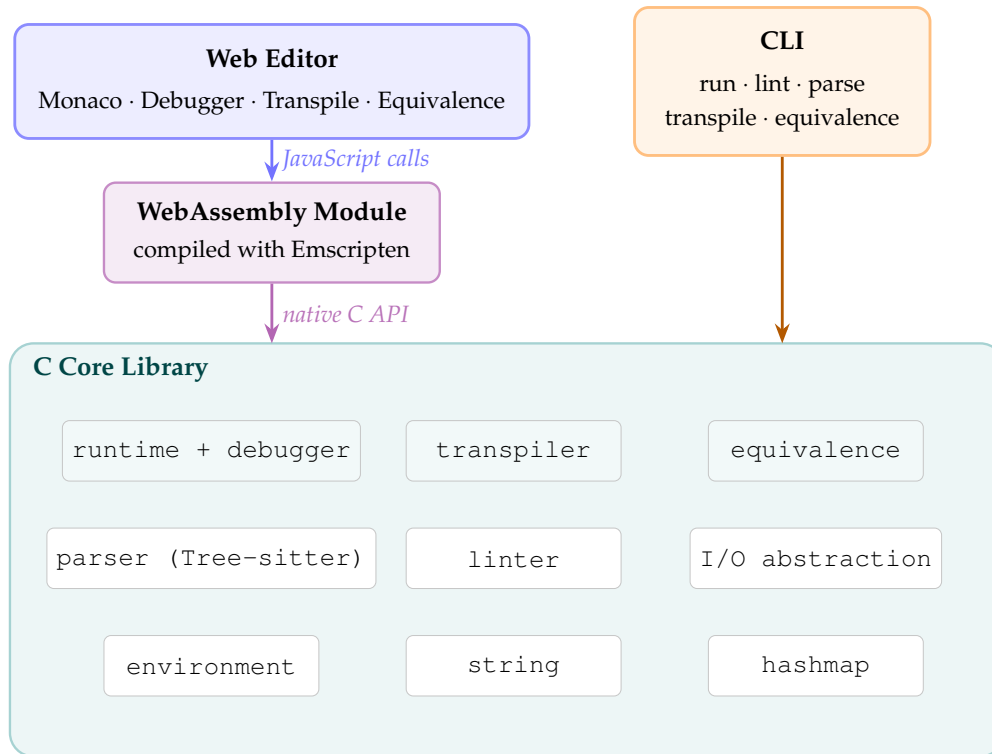
This chapter describes the design and implementation of the Pseudo++ toolchain, a complete software pipeline for writing, executing, debugging, and transpiling Romanian-style pseudocode as taught at the Romanian high-school level. The toolchain is written in C (core library) and JavaScript (web editor, Tree-sitter grammar), and is distributed as a native command-line binary and a browser-based editor that comes bundled with the core implementation compiled to WebAssembly.

4.1 System Architecture

The Pseudo++ toolchain is structured as four layers of independent modules, each with a narrow public interface. Figure 4.1 shows the compile-time dependency graph of all modules grouped by layer.

The modules and their responsibilities are:

- linter** Normalises the raw source text before parsing: it replaces Unicode mathematical symbols (e.g. $\leq$, $\neq$) with their ASCII equivalents and re-indent the code structurally.
- parser** Wraps the Tree-sitter incremental parser with the custom `pseudo` grammar, exposes the resulting concrete syntax tree (CST), and provides helper utilities for navigating and extracting text from CST nodes.
- runtime** Implements the tree-walking interpreter. It subdivides into three sub-modules: `value` (runtime values and arithmetic), `environment` (variable store), and `interpreter` (execution engine with explicit stack).
- debugger** Extends the interpreter with snapshot-based reversible execution, variable inspection, and JSON serialisation of runtime state for the browser front end.



transpiler Converts the CST to equivalent C, C++, or Pascal source, performing a two-pass analysis: first collecting variable types, then emitting the target code.

equivalence Implements loop-equivalence transformations, converting a loop construct of one kind (e.g. `for`) into any of the other three (`while`, `do-while`, `repeat`) while preserving semantics.

io Provides an abstract I/O interface with two concrete backends: `io_stdio` for the CLI and `io_buffered` for the WebAssembly build.

wasm An Emscripten bridge that exports the runtime, transpiler, and equivalence module as `EMSCRIPTEN_KEEPALIVE` C functions callable from JavaScript.

cli The command-line entry point, supporting `run`, `transpile`, `debug`, `parse-tree`, and `equivalence` sub-commands.

4.2 Language Grammar

The grammar for the Pseudo++ language is specified as a Tree-sitter [2] grammar in JavaScript (`grammar.js`). It defines the complete syntax of the Romanian high-school pseudocode dialect, following the conventions used in the Romanian baccalaureate exam [13].

4.2.1 Statement Forms

The top-level production program is a repetition of `stmt` nodes. Each statement is one of:

- **Assignment** (`assign`): `x <- expr` — assigns the value of *expr* to variable *x*.
- **Swap** (`swap`): `x <-> y` — exchanges the values of *x* and *y*.
- **Read** (`read`): `citeste x, y, z` — reads values from standard input.
- **Write** (`write`): `scrie expr1, expr2` — writes values to standard output.
- **Conditional** (`if`): `daca cond atunci ... [altfel ...] sf.`
- **Counting loop** (`for`): `pentru i <- start, end[, step] executa ... sf.`
- **Pre-condition loop** (`while`): `cat timp cond executa ... sf.`
- **Post-condition loop** (`do.while`): `executa ... cat timp cond.`
- **Repeat-until loop** (`repeat`): `repeta ... pana cand cond.`
- **Compound statement** (`multi_stmt`): two or more simple statements separated by semicolons, used where only a single statement is syntactically permitted.

4.2.2 Expression Hierarchy

Expressions follow a standard precedence hierarchy encoded via Tree-sitter's `prec.left` operator:

Level	Operator(s)	Node type
1 (lowest)	SAU / sau	or_expr
2	SI / si	and_expr
3	NOT / not	not_expr
4	= != < <= > >=	compare_expr
5	+ -	add_expr
6	* / %	mul_expr
7 (highest)	-(x), [x], $\sqrt{x}$	neg_expr, floor, sqrt_expr

Atoms (`atom`) are integer literals, floating-point literals, string literals, or identifiers. Parenthesised sub-expressions (`paren`) reset precedence. The floor operator uses bracket notation (`[expr]`), which maps to integer truncation, while the square-root operator uses the radical symbol ($\sqrt{}$).

4.3 Source Normalisation

The official Romanian high-school pseudocode notation [13], as it appears in textbooks and on bacalaureate exam papers, uses a rich set of Unicode characters: relational symbols ($\leq$, $\neq$, $\geq$), assignment and swap arrows ($\leftarrow$, $\leftrightarrow$), box-drawing characters for structural indentation (box-drawing bar), a filled square (■) as the block-closing symbol instead of the keyword `sf`, and letters with Romanian diacritics. This notation is the *canonical* form of the language — it is not incidental variation but the deliberate typographic standard taught in schools.

Formalising this notation directly into a parser grammar would tightly couple the grammar to a specific Unicode encoding and make the language impractical to type in a plain-text environment. The toolchain therefore introduces a deliberate split: the *source language* that users write (and that the parser accepts) uses ASCII-compatible tokens, while the *display language* visible in the web editor renders those tokens back into the canonical Unicode form via ligatures and glyph substitutions. This rendering is purely visual and optional: a toggle button in the editor enables or disables it, leaving the underlying source file unchanged in either case.

The linter is the bridge between the two representations. It accepts source in either form — canonical Unicode or ASCII — and produces the normalised ASCII form consumed by the parser. The normalisation pipeline consists of two passes:

Pass 1 — Symbol Substitution. The linter maintains a static hash map that maps Unicode byte sequences to their ASCII equivalents (Table 4.1). The pass iterates through the source byte-by-byte, scanning the substitution map at each position for the longest matching key (a greedy, maximum-munch strategy). When a match is found the corresponding replacement string is appended to the output and the cursor advances by the match length; otherwise the current byte is copied verbatim.

Canonical input	Category	ASCII output
$\leq, \neq, \geq$	Relational symbols	<code><=, !=, >=</code>
$\leftarrow$	Assignment arrow	<code><-</code>
$\leftrightarrow$	Swap arrow	<code><-></code>
■ (U+25A0)	Block-close square	<code>sf</code>
box-drawing bar (U+2502)	Structural indent	<code>4 spaces</code>
<code>\, „</code>	Curly quotes	<code>"</code>
<code>ă, â, î, ș, ț</code>	Romanian diacritics	<code>a, a, i, s, t</code>

Table 4.1: Selected substitution rules applied by the linter in Pass 1. The canonical Unicode notation is the standard form used in Romanian high-school textbooks and exam papers; the ASCII output is the form accepted by the Tree-sitter grammar.

Pass 2 — Structural Re-indentation. After symbol substitution the source may have inconsistent indentation, particularly when the box-drawing indentation markers have been replaced by spaces. The second pass strips all leading whitespace from every line and recomputes indentation from scratch based on a set of *indent triggers*:

- *Indent-after* lines — lines whose last word is `atunci`, `executa`, `altfel`, or whose first word is `repeta` — increase the indentation level for the next line.
- *Dedent-before* lines — `sf` alone, `altfel` as the first word, and the closing `pana` `cand` of a repeat loop — decrease the indentation level before the line is emitted.

The result is a clean, consistently-indented ASCII source that the Tree-sitter parser can analyse reliably.

4.4 Parsing and CST Construction

The `parser` module wraps the Tree-sitter C API. Its public interface centres on an opaque `parser_t` struct that owns a `TSParser` (the Tree-sitter parser instance), a `TSTree` (the last parse result), and a copy of the normalised source string.

Parsing. `parser_parse()` first copies the source, then calls `ts_parser_parse_string()`. Tree-sitter guarantees a result tree even on malformed input, using `ERROR` and `MISSING` synthetic nodes to indicate problem sites. After the C API call, the function performs a depth-first walk of the resulting tree looking for the first such error node; if one is found, it reports the row and column of the error and constructs a human-readable diagnostic message.

Node Helpers. Because the interpreter and transpiler perform extensive tree traversal, `parser.c` also provides a set of helper functions used throughout the codebase:

- `parser_node_is_type(node, type)` — string-compares the node's grammar type against a constant.
- `parser_child_by_field(node, field)` — wraps `ts_node_child_by_field_name`, used to retrieve named children such as `condition`, `left`, `right`, `var`, `start`, `end`, and `step`.
- `parser_node_text(parser, node)` — extracts the source substring corresponding to a node by using its start and end byte offsets.

- `parser_pretty_tree()` / `parser_debug_tree()` — produce human-readable renderings of the CST for diagnostic purposes.

4.5 Interpreter

4.5.1 Value Representation

Runtime values are represented by a heap-allocated, tagged-union type `value_t`:

```
struct value {
    value_type_t type;          /* VALUE_INT | VALUE_FLOAT | VALUE_STRING */
    union {
        int64_t    int_val;
        double     float_val;
        string_t*  string_val;
    };
};
```

The three types cover the full range of values expressible in Romanian bacculaureate pseudocode. The following design decisions govern arithmetic:

- **Type promotion.** When one operand is `VALUE_FLOAT` and the other is `VALUE_INT`, the integer is promoted via `value_to_float()` before the operation is applied.
- **Division always yields a float.** The `/` operator always returns a `double`, mirroring the mathematical convention used in Romanian textbooks. Integer division is expressed explicitly with the floor operator `[a/b]`.
- **Modulo operates on integers.** The `%` operator truncates both operands to `int64_t` before computing the remainder.
- **Square root returns an integer when the result is exact.** If `sqrt(d) == floor(sqrt(d))`, the result is returned as `VALUE_INT`.
- **String concatenation.** The `+` operator is overloaded: when both operands are strings it performs concatenation; mixed string/numeric operands are a type error.

Logical operations (`SI`, `SAU`, `NOT`) operate on the *truthiness* of any value: a numeric zero and an empty string are false; everything else is true. They always return `VALUE_INT` (1 or 0).

All arithmetic functions follow the same error-reporting convention: an optional `value_error_t* err` output parameter is set to `VALUE_OK`, `VALUE_ERR_TYPE`, `VALUE_ERR_DIV_ZERO`, or `VALUE_ERR_NEGATIVE_SQRT`.

4.5.2 Execution Environment

Variables are stored in a flat `environment_t`, implemented as an open-addressing hash table with linear probing. The table uses three slot states (`SLOT_EMPTY`, `SLOT_OCCUPIED`, `SLOT_DELETED`) to support deletion without disrupting existing probe chains. The FNV-1a hash function [6] is used for string keys:

```
static uint64_t hash_string(const char* str) {
    uint64_t hash = 14695981039346656037ULL;
    while (*str) {
        hash ^= (uint64_t)(unsigned char)(*str++);
        hash *= 1099511628211ULL;
    }
    return hash;
}
```

The table starts with capacity 16. When the combined occupancy of `OCCUPIED` and `DELETED` slots exceeds a 70% load factor, `env_resize()` doubles the capacity and rehashes all live entries. Ownership of values is transferred on set: the table takes ownership of the `value_t*` passed to `env_set()`, and the previous value (if any) is destroyed in place.

An important semantic detail is that Pseudo++ variables are *implicitly initialised*: reading an undeclared variable returns an integer zero and also creates the binding, matching the convention in Romanian textbook examples where variables are used before being formally assigned.

4.5.3 Expression Evaluation

Expressions are evaluated by a recursive `eval_expr()` function that walks the CST nodes produced by Tree-sitter. For binary operators (`add_expr`, `mul_expr`, `compare_expr`) the function extracts the named children `left`, `op`, and `right`, evaluates the sub-expressions recursively, reads the operator text, and dispatches to the appropriate `value_*` arithmetic function.

Short-circuit evaluation is implemented for the logical connectives: `or_expr` returns 1 immediately if the left operand is truthy without evaluating the right; `and_expr` returns 0 immediately if the left operand is falsy.

After each binary expression evaluation the runtime checks whether an error flag was set; if so, it transitions to the `EXEC_ERROR` state and stores the error message, halting further execution.

4.5.4 Control Flow

The interpreter uses an *explicit execution stack* rather than recursion. This design choice serves two goals: it prevents stack overflow for deeply nested pseudocode, and it allows the debugger to capture and restore the full execution state as a data structure.

The stack holds up to `MAX_STACK_DEPTH` frames. Each frame (`exec_frame_t`) records:

- the `frame_type_t` (one of `FRAME_PROGRAM`, `FRAME_BLOCK`, `FRAME_IF`, `FRAME_FOR`, `FRAME_WHILE`, `FRAME_DO_WHILE`, `FRAME_REPEAT`);
- the corresponding CST node;
- a phase integer and `child_idx` counter tracking which part of the construct has already been executed;
- loop-specific fields (`loop_current`, `loop_end`, `loop_step`, `loop_var`) for the `for` frame.

A compile-time dispatch table maps each frame type to its step function:

```
static const frame_step_fn k_step_fns[] = {
    [FRAME_PROGRAM]   = step_program,
    [FRAME_IF]        = step_if,
    [FRAME_FOR]        = step_for,
    [FRAME_WHILE]      = step_while,
    [FRAME_DO_WHILE]   = step_do_while,
    [FRAME_REPEAT]     = step_repeat,
    [FRAME_BLOCK]      = step_block,
};
```

Each step function advances the frame by one *phase* and returns a Boolean indicating whether a *visible action* occurred. Visible actions include executing a simple statement, evaluating a loop condition, and terminating the program; invisible actions are internal transitions such as resetting `child_idx` at the start of a new loop iteration.

The public API exposes three execution modes:

- `runtime_step()` — advances to the next visible action. Used by the step-by-step debugger.
- `runtime_run()` — runs to completion or the next input request via a tight loop with no overhead. Used by both the CLI `run` command and the web editor's Run button.

The `EXEC_NEEDS_INPUT` state suspends execution when a `read` statement encounters an unavailable token. The web front end supplies the missing token through `io_buffered` and calls `runtime_resume()` to continue.

4.6 Debugger

The debugger extends the interpreter with step-by-step execution and runtime state inspection, following the snapshot-based approach to reversible debugging described by Lewis [10] and surveyed by Engblom [7]. The web editor exposes three operations: **Step Into** (advance one visible action), **Continue** (run to completion or the next input request), and **Stop**. After each step, the front end displays the currently executing line, the full variable table, which variables changed since the last step, and — when the step involved evaluating a condition — the condition text and its boolean result.

This is made possible by a *snapshot-based* architecture in the C runtime that captures the complete interpreter state before every step.

Snapshot structure. A `runtime_snapshot_t` captures the complete interpreter state at a given execution point:

- All variable bindings (name, value, and type) as a `var_info_t` array.
- The full execution stack: for each frame, its type, phase, `child_idx`, loop counters, and the *start byte* of the corresponding CST node.
- The auxiliary fields `read_var_index`, `has_pending_read`, `current_line`, and `exec_state`.

CST nodes are identified by their `start_byte` offset (a `uint32_t`) rather than by pointer. On restoration, `find_node_by_start()` locates the matching node in the unchanged CST. Because the CST is never modified after parsing, this identity scheme is safe and avoids serialising the full tree.

Circular buffer. Snapshots are stored in a fixed-size circular buffer of `MAX_SNAPSHOTS` entries, giving the JavaScript layer a history of past states it can replay without re-executing the program from the beginning. When the buffer is full, the oldest snapshot is evicted.

JSON serialisation. The function `runtime_get_variables_json()` serialises the current variable table to a JSON array of `{name, value, type}` objects, with proper JSON string escaping. This JSON string is the primary mechanism by which the WebAssembly runtime communicates variable state to the JavaScript front end on each step.

4.7 Transpiler

4.7.1 Code Generation for C, C++, and Pascal

The transpiler converts a normalised Pseudo++ CST to compilable source code in C, C++, or Pascal. Because these three languages differ in significant ways — type declarations, operator syntax, string literals, and logical keyword precedence — the transpiler abstracts all language-specific choices into a `lang_ops_t` record populated once at construction time:

```
typedef struct {
    const char* assign_op;    /* "=" (C/C++) or "!=" (Pascal) */
    const char* eq_op;       /* "==" or "=" */
    const char* ne_op;       /* "!=" or "<>" */
    const char* or_op;       /* "||" or "or" */
    const char* and_op;      /* "&&" or "and" */
    const char* not_kw;      /* "!" or "not " */
    const char* floor_open;  /* "(int)(" or "trunc(" */
    const char* floor_close; /* ") " */
    bool is_pascal;
    bool is_cpp;
} lang_ops_t;
```

Transpilation proceeds in two passes:

Pass 1 — Variable Type Collection. `collect_vars()` performs a depth-first walk of the CST. For each assignment or read statement it records the inferred type of the target variable in a hash map (`var_types`). The inference rules are:

- If the right-hand-side expression contains a string literal, the variable is typed as `string`.
- If it contains a floating-point literal (a number with a decimal point), it is typed as `double`.
- Otherwise it is typed as `int`.

The first assignment wins for the purpose of code generation; subsequent assignments may change the value but not the declared type in the target language. This is a transpiler-level heuristic, not a language restriction: the interpreter is fully dynamically typed and allows a variable to hold values of different types at different points in execution.

Pass 2 — Code Emission. The emission pass walks the CST recursively via `gen_stmt()`, using the `lang_ops_t` record to emit the correct syntax for each construct. Several language-specific concerns deserve mention:

- **C variable declarations.** C requires variables to be declared before use. Rather than adding a separate declaration section, the transpiler hoists declarations *inline*: the helper `hoist_vars()` scans each block before emitting its first statement and pre-declares any variable first assigned or read in that block. A `declared_vars` hash map prevents duplicate declarations. C strings are emitted as `char name[256]`; C++ strings use `std::string`.
- **Pascal variable section.** For Pascal, variable declarations are collected globally during Pass 1 and emitted in a `var` block at the top of the program before the `begin` keyword.
- **Pascal logical operator precedence.** In Pascal, `and` and `or` bind *more tightly* than comparison operators, unlike C where `&&` and `||` bind less tightly. To preserve semantics, the transpiler wraps each operand of an `and_expr` or `or_expr` in parentheses when targeting Pascal.
- **Pascal string literals.** String literals are enclosed in single quotes in Pascal. The transpiler re-encodes any embedded single quotes using Pascal's doubling convention (`' '`).

4.7.2 Loop Equivalence Transformations

The loop equivalence module implements a pedagogically important feature: given any loop in a Pseudo++ program, it rewrites that loop into an equivalent form using a different loop construct, grounded in the interconvertibility established by Böhm and Jacopini [3] and the structured programming principles of Dijkstra [5]. This allows students to practise the mechanical equivalence proofs required at the baccalaureate exam.

The module accepts a source string, a 0-indexed target line identifying the loop to transform, and the desired output construct (`FOR`, `WHILE`, `DO.WHILE`, or `REPEAT`). It then:

1. Runs the source through the linter and parser to obtain a CST.
2. Locates the loop node at the specified line via DFS.
3. Extracts the loop header fields (*var*, *start*, *end*, *step*, *condition*) and body text from the CST and the raw source string.
4. Constructs the target loop form by selecting the appropriate skeleton and filling in the extracted fields.
5. Splices the new loop text back into the original source, preserving all preceding and following code unchanged.

Indentation is preserved by detecting the loop's own indentation level from the source and replicating it in the emitted loop.

4.8 I/O Abstraction and WebAssembly Integration

4.8.1 I/O Abstraction

The runtime communicates with the outside world exclusively through an abstract `io_t` interface:

```
typedef struct {  
    const char* (*read)(io_t*);    /* returns next token or NULL */  
    void (*write)(io_t*, const char*);  
    void (*destroy)(io_t*);  
} io_ops_t;  
  
struct io { io_ops_t ops; };
```

Two concrete implementations are provided:

- `io_stdio` — delegates directly to `scanf` and `printf`, used by the CLI.
- `io_buffered` — maintains an input queue and an output queue. JavaScript pushes input tokens by calling `pseudo_push_input()` (a WASM export) and pulls output by calling `pseudo_pull_output()`. When the input queue is empty, `read()` returns `NULL`, causing the runtime to transition to `EXEC_NEEDS_INPUT`.

4.8.2 WebAssembly Bridge

The WASM build compiles the entire C library with Emscripten [17], targeting the WebAssembly instruction format [9]. The bridge module (`wasm/bridge.c`) instantiates a singleton `runtime_t` and a singleton `io_buffered`, then exposes the

toolchain to JavaScript via `EMSCRIPTEN_KEEPALIVE`-annotated functions, which are exported by name from the resulting `.wasm` module. The exported surface includes functions for initialisation, source loading, single-step and run-to-completion execution, input supply, output retrieval, variable inspection, transpilation, and loop equivalence.

Separating the I/O layer behind an interface means that the *same* runtime and debugger C code is used in both the CLI and the browser — only the I/O backend differs.

4.9 Web Editor

The web editor is a single-page application built around the Monaco Editor [11], the same editing component that powers Visual Studio Code. For the `pseudocode` language, the editor registers:

- A **Monarch tokeniser** [12] that classifies tokens into keyword, operator, string, number, comment, and identifier categories for syntax highlighting.
- A **language configuration** that specifies bracket pairs (for auto-closing), the line-comment token, and auto-indentation rules.

At runtime the JavaScript application loads the compiled WASM module and communicates with it through the exported C functions. The interface provides:

- **Run** — compiles the editor content, sends it to the WASM runtime, and streams output tokens to the output pane.
- **Step / Step Over / Step Back** — calls the corresponding WASM step function, then queries the runtime for the current line, variable table (JSON), and condition information to update the highlighted line and the variable inspector panel.
- **Transpile** — calls the WASM transpiler with the selected target language and displays the result in a read-only Monaco pane.
- **Loop Equivalence** — identifies the loop under the cursor and passes it to the equivalence module.

4.10 VSCode Extension

The Visual Studio Code extension provides the same capabilities as the web editor but integrated into the developer’s local environment. It bundles the native CLI

binary (compiled for the target platform) and invokes it as a child process. The extension contributes:

- Syntax highlighting, through a TextMate grammar derived from the Monarch tokeniser rules.
- A **Run Pseudo++** command that executes the active file with the bundled CLI and streams output to a dedicated terminal.
- A **Transpile** command that runs the CLI's `transpile` sub-command and opens the result in a new editor tab.

Chapter 5

Evaluation

This chapter evaluates the correctness and performance of the Pseudo++ toolchain. The evaluation is structured into three tiers: unit tests for the linter module, integration tests that execute official Romanian baccalaureate problems end-to-end, and semantic-preservation tests for the loop equivalence module. Transpiler correctness is assessed by compiling the generated C code with GCC and comparing its output against the known-correct interpreter results.

5.1 Testing Methodology

The test suite is divided into two directories:

- `test/` — C unit tests compiled and run by the build system (`make test`). Each test file exercises a single module in isolation.
- `int-test/` — integration tests organised into two sub-trees:
 - `int-test/bac/year/antrenamentN/` — one directory per baccalaureate training problem, each containing the canonical source (`src.pseudo`), the linter-normalised source (`cleaned-src.pseudo`), an input file (`input.txt`), and the expected output (`expected-output.txt`).
 - `int-test/equivalence/A_to_B/` — one directory per directed loop conversion, each containing the source (`src.pseudo`), the target loop type (`target.txt`), the 0-based line index of the loop (`loop_line.txt`), optional input (`input.txt`), and the expected execution output (`expected-output.txt`).

All integration tests are run by invoking the `pseudo` CLI binary directly with the normalised source and the provided input. The output is compared byte-for-byte against the expected file.

5.2 Unit Testing

The linter module is covered by 39 unit tests in `test/test_linter.c`. Each test constructs a short input string, calls `linter_run()`, and asserts the normalised output against a known-correct string using a simple `ASSERT_STR_EQ` macro. The cases cover:

- Symbol substitution for all entries in the substitution table (assignment arrow, relational operators, block-close square, box-drawing bar, smart quotes, Romanian diacritics).
- Greedy, maximum-munch ordering: ensuring that a two-byte Unicode sequence is matched as a single token rather than as two separate ASCII characters.
- Edge cases: empty input, a string consisting entirely of replacement sequences, and replacements at the start and end of the input.
- Keyword normalisation: the diacritic variants of Romanian keywords (`citește` / `citește`, `cât timp`, `până când`, etc.) are mapped to their ASCII equivalents.
- Structural re-indentation: pipe-indented and box-drawing-indented source is correctly flattened and re-indented.

All 39 unit tests pass.

5.3 Integration Testing on Baccalaureate Examination Problems

5.3.1 Dataset Description

The integration test dataset consists of 53 pseudocode problems drawn from the official Romanian baccalaureate training sets published by the Ministry of Education for examination years 2020 through 2025 [13]. Table 5.1 shows the breakdown by year.

The problems cover the full range of constructs present in the dialect: all four loop forms, nested conditionals, string and integer arithmetic, the floor-division and modulo operators, and multi-variable `citește` statements. Each problem is supplied with one or more input values and a single expected output string.

Year	Problems
2020	20
2021	12
2022	5
2023	5
2024	5
2025	6
Total	53

Table 5.1: Baccalaureate integration test dataset by year.

5.3.2 Results

The interpreter correctly produces the expected output for all **53 out of 53** problems (100 %). No failures were observed across any of the six examination years.

5.4 Transpiler Correctness Verification

Transpiler correctness is verified for the C target using the 32 problems from the 2020 and 2021 baccalaureate sets (the subset for which reference input-output pairs were available at transpiler evaluation time). For each problem the following pipeline is executed automatically:

1. Run `pseudo_transpile c src` to produce a C source file.
2. Compile the generated C with `gcc -lm`.
3. Execute the compiled binary with the same input as the interpreter test and compare its standard output against the reference.

All **32 out of 32** problems (100 %) produce compilable C code whose output matches the interpreter reference exactly. GCC reported no compilation errors or warnings for any of the generated files.

5.5 Loop Equivalence Semantic Preservation

The equivalence module is evaluated with 9 directed conversion tests, one for each ordered pair among the four loop constructs that changes the loop type:

Source	Target
for	while
for	do-while
for	repeat
while	do-while
while	repeat
do-while	while
do-while	repeat
repeat	while
repeat	do-while

For each test, the module transforms the source loop, writes the result to a temporary file, and the interpreter executes that file with the original input. The execution output is then compared against the expected result produced by running the untransformed program.

All **9 out of 9** transformations (**100 %**) preserve the program’s observable output exactly, confirming that the rewrite rules described in Section 3.4 are implemented correctly.

5.6 Performance Analysis

Execution time was measured by running each of the 32 baccalaureate problems (2020–2021) 100 times in sequence and computing the mean wall-clock time per run on an AMD Ryzen 5 3600 machine running Arch Linux with a release build (`-O3`). The average execution time per problem was **2.4 ms**, with a total time of **108 ms** for all 32 problems combined.

The dominant cost in each run is process start-up and the Tree-sitter parse, not the interpreter loop itself. For the web editor the start-up cost is paid once when the WASM module is loaded; subsequent invocations reuse the already-loaded runtime, making the in-browser execution latency significantly lower than the CLI benchmark suggests.

At 2.4 ms per program the interpreter is fast enough for interactive use: a student pressing the **Run** button receives output well within the threshold of perceived immediacy (<100 ms for a single-line response). The step-by-step debugger incurs additional overhead per step due to snapshot serialisation, but this overhead is negligible compared to the human reaction time between debugger clicks.

5.7 Discussion

The evaluation demonstrates that the Pseudo++ toolchain correctly handles all 53 baccalaureate problems tested, covering examination years 2020 through 2025. The transpiler generates valid, warnings-free C code for every problem in the verification subset. Loop equivalence transformations preserve observable output in all nine directed-conversion cases.

Several limitations should be noted. First, the test dataset consists exclusively of single-function programs with simple integer arithmetic; the dialect does not support user-defined functions or arrays, so these constructs were not exercised. Second, transpiler verification was performed only for the C target; the C++ and Pascal backends share the same emission logic but were not independently benchmarked. Third, the performance figures reflect CLI execution on a desktop machine; browser performance under the WebAssembly runtime was not formally measured, though informal testing confirms sub-100 ms response times for all tested problems.

Chapter 6

Conclusions

6.1 Summary

6.2 Limitations

6.3 Future Work

Bibliography

- [1] Max Brunsfeld. An incremental parsing system for programming tools. Master's thesis, Massachusetts Institute of Technology, 2018.
- [2] Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. In *Proceedings of the USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, 2018.
- [3] Corrado Böhm and Giuseppe Jacopini. Flow diagrams, turing machines and languages with only two formation rules. *Communications of the ACM*, 9(5):366–371, 1966.
- [4] Martin C. Carlisle, Terry A. Wilson, Jeffrey W. Humphries, and Steven M. Hadfield. RAPTOR: Introducing programming to non-majors with flowcharts. In *Proceedings of the 36th ACM SIGCSE Technical Symposium on Computer Science Education*, pages 52–56, 2005.
- [5] Edsger W. Dijkstra. Go to statement considered harmful. *Communications of the ACM*, 11(3):147–148, 1968.
- [6] Donald Eastlake, Tony Hansen, Glenn Fowler, Landon Curt Noll, and Kiem-Phong Vo. The FNV non-cryptographic hash algorithm. Internet-draft, Internet Engineering Task Force, 2019. <https://datatracker.ietf.org/doc/html/draft-eastlake-fnv>.
- [7] Jakob Engblom. A review of reverse debugging. In *Proceedings of the 2012 System, Software, SoC and Silicon Debug Conference (S4D)*, pages 1–6, 2012.
- [8] Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, and Michael Holman. WebAssembly specification. Technical report, World Wide Web Consortium (W3C), 2019.
- [9] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. Bringing the Web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN*

- Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, 2017.
- [10] Bil Lewis. Debugging backwards in time. In *Proceedings of the Fifth International Workshop on Automated and Algorithmic Debugging (AADEBUG)*, 2003.
- [11] Microsoft Corporation. Monaco editor. <https://microsoft.github.io/monaco-editor/>, 2016.
- [12] Microsoft Corporation. Monarch: Create declarative language syntax highlighters. <https://microsoft.github.io/monaco-editor/monarch.html>, 2016.
- [13] Ministerul Educației Naționale. Programa școlară pentru disciplina informatică, clasele IX–XII. Technical report, Ministerul Educației Naționale al României, 2004.
- [14] Pablo Novara. PSeInt: A pseudocode interpreter for learning programming. <https://pseint.sourceforge.net/>, 2003.
- [15] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Niki Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. Scratch: Programming for all. *Communications of the ACM*, 52(11):60–67, 2009.
- [16] Jeannette M. Wing. Computational thinking. *Communications of the ACM*, 49(3):33–35, 2006.
- [17] Alon Zakai. Emscripten: An LLVM-to-JavaScript compiler. In *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 301–312, 2011.